**BLACK PEN** Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

# Binary Search

**Name** _____ **Class** _____ **Date** _____ **Time** _____ **Teacher** _____

## Do Now - Four retrieval questions

BLACK PEN: answer four questions before checking the answers.

1. Does linear search require a sorted list?

2. What is decomposition?

A) Splitting a problem into parts
B) Removing all useful details
C) Converting text to binary
D) Testing only the final program

3. "Every local variable can always be accessed directly from every other subprogram." Correct this. Explain why.

4. Several procedures share a global score. "A global variable can be read only in one subprogram." Correct this. What risk does sharing it create?

## Today's objectives

Explain the steps and required conditions for binary search. Apply the algorithm or method to supplied data.

## Key vocabulary - Read, cover, recall

**Binary search**
Halves a sorted search range after each unsuccessful comparison.

**Sorted prerequisite**
Values must be in a known order.

**Midpoint**
Middle index of current search interval.

**Low/high**
Inclusive bounds of the current search interval.

**Write one key term and its meaning in your own words:**

**BLACK PEN** Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

# Learn and practise

## New learning - Key ideas

| | |
|---|---|
| **Sorting is essential** | Binary search only works correctly when the array is ordered by the required comparison. |
| **Start with a range** | Keep low and high indexes of the active range, then calculate midpoint using integer division. |
| **Discard half** | If the target is larger than midpoint value, set low = mid+1; if smaller, set high = mid-1. |
| **Stop correctly** | Return the midpoint when equal; stop unsuccessful when low becomes greater than high. |

## I Do - One worked example

**Trace binary search for 23 in [3,7,11,15,19,23,27,31]. Show low, mid and high. [5 marks]**

1. Check precondition

2. The list is sorted; low=0 and high=7.

3. mid=(0+7) DIV 2=3; value 15 is less than 23, so low becomes 4.

4. mid=(4+7) DIV 2=5; value 23 matches. Found at index 5. [5]

## We Do - Guided questions

BLACK PEN: use the model, then explain your answer.

1. Find 41 in [5,9,13,21,29,33,37,41,45] using binary search. [5]

2. Explain why binary search cannot be relied upon for [9,2,7,1,5]. [3]

3. Search for 6 in [2,4,8,10]. Explain when it stops. [4]

**BLACK PEN** Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

# Think and write

## Code First - Predict > Write > Test > Improve

**YOUR TASK: Complete the midpoint assignment for a sorted list of bit weights.**

```
array weights = [1,2,4,8,16,32,64]
low = 0
high = 6
mid = ...
print(weights[mid])
```

BLACK PEN: predict or complete the code; test, then improve.

For extended answers, continue on lined paper if needed.

## You Do - Independent exam questions

Q2. A sorted array is [1,5,9,13,17,21,25]. Trace the mid indexes when searching for 21 and give its final index. [4]

Q3. State the prerequisite for binary search and explain its importance. [3]

Q4. Give the condition that shows binary search failed to locate an absent value. [2]

## Exit Ticket - True or False?

Circle T or F before reviewing Page 4.

1. Binary search can safely discard half only when data is sorted. ☐ T ☐ F
2. Binary search only works correctly when the array is ordered by the required comparison. ☐ T ☐ F
3. Keep low and high indexes of the active range, then calculate midpoint using integer division. ☐ T ☐ F

**BLACK PEN** Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

# Review & improve

GREEN PEN: tick correct marking points, then correct errors.

Choose your hardest question to rewrite in full below.

## Do Now - Answers

**Q1** No; it examines items in sequence until a match is found or the list ends. ☐

**Q2** A - Splitting a problem into parts ☐

**Q3** A local variable can be used only within its own subprogram or local scope. ☐

**Q4** Global variables can be used across the program. This risks unwanted changes and makes faults harder to trace. Use arguments and return values to avoid hidden changes. ☐

## Code First - Model

```
array weights = [1,2,4,8,16,32,64]
low = 0
high = 6
mid = (low + high) DIV 2
print(weights[mid])
```

**Output: 8**

## We Do - Check the model

1. low=0, high=8, mid=4 gives 29 [1]; low=5, mid=6 gives 37 [2]; low=7, mid=7 gives 41 [1]; found at index 7 [1]. ☐

2. Data is unsorted [1]; discarding halves depends on ordering [1], so the target might be discarded incorrectly [1]. ☐

3. low0 high3 mid1=4 [1]; low2 mid2=8 [1]; high1 [1], so low2>high1 and target is absent [1]. ☐

## Exit Ticket - Check

**1. TRUE 2. TRUE 3. TRUE**

## You Do - Self-mark

**Q2** Start low0 high6 mid3 value13 [1]; low4 high6 mid5 value21 [1]; target found [1] at index5 [1]. ☐

**Q3** Sorted/ordered array [1]; comparisons with midpoint determine safe half to discard [1], without sorting the target could be excluded incorrectly [1]. ☐

**Q4** low > high [1], meaning no active elements remain [1]. ☐

## Try the question again

**1. Copy the FULL question you found most difficult:**

**2. In GREEN PEN, write a complete new answer:**